

Aula 2 — Setup RN, Manual Estruturado & Unit Testing

"Antes de automatizar UI, e o que vem antes?"

Foco da disciplina: **React Native** (app de referência é RN)

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 28/05/2026

Abertura — Fluxo Fork + PR (~15min)

Antes do conteúdo técnico, vamos cobrir **como entregar atividades via GitHub**.

 Slides + guia escrito no repo da disciplina:

- `slides/intro-fluxo-pr-github.pdf`
- `COMO_ENTREGAR.md` (raiz do repo)

Cobre: fork → clone → branch → commit → push → PR → link Canvas.

Termina com **demo ao vivo** num repo dummy.

| Garantir que ninguém trave na entrega da próxima atividade.

Recap da Aula 1

- Ecossistema mobile: 3M apps, 77% abandono
- Pirâmide de Knott vs Cohn
- Tipos de teste mobile (10 categorias)
- Real device vs emulator vs cloud farm

Atividade 1 — Análise de Cobertura entregue?

Agenda da aula (3h30)

0. **Abertura (15min)** — fluxo Fork + PR
1. **Bloco 1 (75min)** — Setup RN + Manual estruturado (SBTM) + exercício exploratório
2. **Intervalo (15min)**
3. **Bloco 2 (90min)** — Hands-on: testes RN (Jest + RNTL) + cobertura + mutation
4. **Wrap-up (15min)** — Atividade 2

Objetivos de aprendizagem

- **Configurar** ambiente React Native (Node, Metro, simulators iOS/Android)
- **Estruturar** testes manuais com SBTM (Bach)
- **Aplicar** heurísticas FEW HICCUPPS (Bolton)
- **Implementar** testes unitários RN — Jest (domain puro) + React Native Testing Library
- **Aplicar** test doubles (Meszaros) com `jest.fn` / `jest.mock`

Setup ambiente RN — checklist

Guia oficial Expo — **escolhe teu OS** (macOS / Windows / Linux) + iOS/Android:

docs.expo.dev/get-started/set-up-your-environment

```
# 1. Node 22 LTS — só precisa disso pra testar
#   macOS/Linux: nvm install 22   |   Windows: nvm-windows ou fnm
node -v    # v22.x

# 2. Criar app + rodar testes (Jest já vem no template)
npx create-expo-app@latest MeuApp
cd MeuApp && npm test
```

Tarefa	macOS	Windows	Linux
Unit test (hoje) — só Node	✓	✓	✓
Build Android (E2E)	✓	✓	✓
Build iOS (E2E)	✓	✗	✗

Vantagem QA RN: unit (Jest) roda em **qualquer OS** — sem simulador, token nem rede. Só Node. Build nativo (iOS exige Mac) fica pras Aulas 3-4 (E2E).

Manual testing estruturado — por que importa

Na **Atividade 1** vocês já usaram FEW HICCUPPS + SBTM pra redigir casos. Aqui **formalizamos** a técnica — não é repetição, é aprofundar.

Em mobile, **manual ainda é peça central** porque automação não cobre:

- Sensação de UX (responsividade tátil)
- Comportamento em conectividade variável
- Interação com sensores
- Permissões iOS/Android dialogs
- Comportamento em background/foreground transitions
- Acessibilidade real

Mas manual sem estrutura é caos. SBTM resolve.

Session-Based Test Management (SBTM)

Proposto por **James Bach**.

Sessão: unidade de tempo dedicada (60–120min).

Estrutura:

1. **Charter** — missão da sessão em 1 frase
2. **Tester** — quem executa
3. **Duração** — alvo (60min, 90min, 120min)
4. **Notas** — log do que fez, o que viu
5. **Bugs** — issues levantadas
6. **Issues** — perguntas/debates
7. **Debriefing** — discussão pós-sessão

Estrutura sem virar script. Foco com flexibilidade.

Charter — exemplo

CHARTER:

Explorar fluxo de checkout do app de e-commerce
focando em: cupons, métodos de pagamento alternativos
e cancelamentos antes de confirmação.

Objetivo: descobrir bugs de UX/funcionais.

Duração: 90min.

Charter dá foco. Tester explora dentro do escopo, livre pra perseguir descobertas.

Pitfall: exploratório **sem charter** vira clicar aleatório — sem cobertura, sem rastro.

Heurísticas — FEW HICCUPPS

Michael Bolton. Heurística pra cobertura sistemática:

- **Familiar** — conformidade com padrões da plataforma
- **Explicit** — comportamento explicitamente especificado
- **World** — comportamento esperado pelo usuário real
- **History** — comportamento histórico do produto
- **Image** — imagem da empresa/produto
- **Comparable products** — comparáveis no mercado
- **Claims** — promessas marketing/docs
- **User desires** — desejos não declarados
- **Purpose** — propósito do produto
- **Product** — coerência interna
- **Statutes** — leis, regulações

Whittaker tours

James Whittaker propõe "tours" pelo app:

- **Money tour** — onde o produto gera receita
- **Landmark tour** — features principais
- **Antisocial tour** — entradas inválidas, edge cases
- **Obsessive tour** — repetir mesma ação ad infinitum
- **All-nighter tour** — deixar app aberto 24h+
- **FedEx tour** — como dado entra e sai do app

Use como prompt para sessões SBTM.

Bug reporting — template eficaz

TÍTULO: [iOS 17] Botão Comprar não responde após cupom expirado

PASSOS:

1. Adicionar item ao carrinho
2. Aplicar cupom expirado (CUPOM_TESTE_2024)
3. Tap no botão "Comprar"

ESPERADO: Mensagem "Cupom expirado, remova para continuar"

ATUAL: Botão não responde, sem feedback visual

REPRO RATE: 5/5 (sempre)

DEVICE: iPhone 14 Pro, iOS 17.5

APP VERSION: 2.3.1 (build 2310)

SCREENCAST: link.loom.com/abc123

Repro rate é diferencial. "Aconteceu uma vez" é hipótese, não bug.

🔍 Exercício relâmpago — Exploratório (10min)

Todos no mesmo app: **Organic Maps** (mapas offline, open-source). Grátis, **sem login**:

| 🍏 App Store · 🤖 Play Store · 🗯 bugs reais e abertos: `github.com/organicmaps/organicmaps/issues`

CHARTER (pronto): *"Buscar um lugar e traçar rota a pé até ele em 8min, via Antisocial tour. Achar bug de UX/funcional."*

1. **1min** — leiam o charter + escolham **1 Whittaker tour** (Antisocial: entradas inválidas)
2. **6min** — explorem. Rodem **FEW HICCUPPS** em ≥ 3 heurísticas. Anotem bugs/issues
3. **3min** — **debrief**: 2 duplas compartilham 1 bug + qual heurística pegou

Por que esse app? É grande mas **não hiper-otimizado** — tem bugs reportados. Você vai achar algo.

Áreas quentes: busca por endereço/POI · traçado e recálculo de rota · detalhe de POI · bookmarks · download de mapa offline.

Registro da sessão (template rápido)

CHARTER: buscar lugar + rota a pé no Organic Maps via Antisocial tour
TOUR: busca vazia, caracteres estranhos, sem internet, POI inexistente

ACHADOS:

- [World] busca de endereço não acha lugar que existe no Google Maps
- [Familiar] gesto de voltar não segue padrão da plataforma
- [Explicit] rota a pé recalcula sozinha sem o usuário pedir

REPRO: 3/3 | DEVICE: [seu cel] | versão do app: [x.y.z]

Mesmo template de bug reporting de antes — agora na prática.

Antes: JS/TS puro vs React

Dois tipos de código num app RN — e cada um testa diferente:

	JS / TS puro	React (RN)
O que é	lógica, dados, regras	componente que desenha UI
Exemplo	<code>favoritesStore</code> , <code>posterUrl</code> , <code>isTokenError</code>	<code><MovieCard/></code> , <code><MovieList/></code>
Tem tela?	não	sim (<code><View></code> , <code><Text></code>)
Pra testar	só Node — rápido	precisa "renderizar" (RNTL)
Custo / flaky	baixo	maior

| Regra de negócio mora no **puro**. UI é só a casca. Testa o puro primeiro.

Unit testing RN — por onde começar

Componentes (RNTL)	poucos · + caros · + flaky
Hooks / estado	renderHook
Domain puro (Jest)	MUITOS · baratos · 0 flaky

↑ base larga = comece por aqui

Tier	Testa o quê	Ferramenta
 Domain puro	função/regra (TS)	Jest — só Node
 Hooks/estado	lógica de estado	renderHook
 Componentes	UI/comportamento	RNTL (precisa render)

Comece pelo domain. Maior ROI, zero flaky, roda só com Node.

Domain layer — Jest puro

Lógica de negócio isolada de React e do nativo. Teste mais barato e estável.

```
// src/domain/cart.ts - função pura
export function cartTotal(items: Item[]): number {
  return items.reduce((sum, i) => sum + i.price * i.qty, 0);
}
```

```
// src/domain/__tests__/cart.test.ts
import { cartTotal } from '../cart';

describe('cartTotal', () => {
  it('soma preço × quantidade', () => {
    const items = [{ price: 10, qty: 2 }, { price: 5, qty: 1 }];
    expect(cartTotal(items)).toBe(25);
  });

  it('retorna 0 pra carrinho vazio', () => {
    expect(cartTotal([])).toBe(0);
  });
});
```

Mockando dependências — jest.mock

Camada **data** depende de API/storage. Mocka a dependência, testa a lógica.

```
// src/data/checkout.ts
import { api } from './api';
export async function checkout(cart: Cart) {
  const res = await api.charge(cart.total);
  return { transactionId: res.id };
}

// src/data/__tests__/checkout.test.ts
import { checkout } from '../checkout';
import { api } from '../api';

jest.mock('../api'); // auto-mock do módulo
const mockedCharge = api.charge as jest.Mock;

it('chama o gateway com o total e devolve a transação', async () => {
  mockedCharge.mockResolvedValue({ id: 'tx-1' }); // stub

  const result = await checkout({ total: 100 });

  expect(result.transactionId).toBe('tx-1');
  expect(mockedCharge).toHaveBeenCalledWith(100); // spy/mock: verifica interação
});
```

Componentes — React Native Testing Library

Testa **comportamento de tela** (o que o usuário vê e toca). `getByText` = o que aparece · `fireEvent.press` = o toque · mock de navegação isola a tela. **O teste mais "cara de QA"**.

```
import { render, screen, fireEvent } from '@testing-library/react-native';
import MovieCard from '../src/components/MovieCard';

const mockNavigate = jest.fn();
jest.mock('@react-navigation/native', () => ({
  useNavigation: () => ({ navigate: mockNavigate }),
}));
const movie = { id: 42, title: 'Matrix', poster_path: '/m.jpg', vote_average: 8.7 };

it('mostra título e nota', () => {
  render(<MovieCard movie={movie} />);
  expect(screen.getByText('Matrix')).toBeTruthy();
  expect(screen.getByText('★ 8.7')).toBeTruthy();
});

it('navega ao tocar no card', () => {
  render(<MovieCard movie={movie} />);
  fireEvent.press(screen.getByText('Matrix'));
  expect(mockNavigate).toHaveBeenCalledWith('Detail', { id: 42, title: 'Matrix' });
});
```

Test doubles (Meszaros)

Tipo	O que faz	Nesta aula
Dummy	Objeto qualquer, nunca usado	—
● Stub	Retorna respostas pré-programadas	✓ mockResolvedValue
● Spy	Stub + grava chamadas	✓ jest.fn()
● Mock	Verifica chamadas com expectativas	✓ toHaveBeenCalledWith
Fake	Implementação simplificada (in-memory DB)	—

● = praticamos hoje no bônus (`jest.mock` da api). Dummy/Fake ficam como conceito.
Não confunda. Stub responde, mock observa.

Cobertura — o que ela mede

Métrica	Mede
Statements	% de instruções executadas
Branches	% de caminhos (if/else , && , ?:)
Functions	% de funções chamadas
Lines	% de linhas executadas

```
function desconto(v: number) {  
  if (v > 100) return v * 0.9;    // teste só do "else" não passa aqui  
  return v;  
}
```

| Teste só do caminho feliz → **Lines 100%, Branches 50%**. Cobertura diz o que rodou, não o que foi verificado.

Cobertura — Jest + Istanbul

```
# RN usa Istanbul por baixo do Jest
jest --coverage
open coverage/lcov-report/index.html
```

```
// jest.config.js – gate de cobertura
module.exports = {
  preset: 'react-native',
  collectCoverageFrom: ['src/**/*.{ts,tsx}', '!src/**/*.d.ts'],
  coverageThreshold: {
    global: { branches: 70, functions: 70, lines: 70, statements: 70 },
  },
};
```

Target inicial: **70%+ em data/domain layer**. UI pode ter cobertura menor.

Lendo o relatório

Tests:	14 passed, 14 total				← quantos CASOS rodaram verde	
-----	-----	-----	-----	-----	-----	-----
File	% Stmt	% Brnc	% Func	% Line	Uncovered	
-----	-----	-----	-----	-----	-----	-----
cart.ts	100	50	100	100	4	← linha não coberta

- **Tests passed** = nº de testes verdes (passaram a asserção)
- **% Coverage** = quanto do **código** os testes tocaram

São coisas **diferentes**: dá pra ter 100 testes verdes cobrindo 10% do código — ou 1 teste cobrindo 100%. **Branch** 50% acima = só 1 caminho do **if** foi testado (linha 4 ficou de fora).

Teste ruim passa e engana

```
// ✗ sem assertion – cobre a linha, não testa nada  
it('renderiza', () => { render(<Tela/>); });  
  
// ✗ tautológico – testa o JS, não o SEU código  
it('soma', () => { expect(2 + 2).toBe(4); });  
  
// ✗ só verifica o mock, ignora o resultado  
expect(api.get).toHaveBeenCalled(); // e o que a função retornou?
```

100% de cobertura + asserts fracos = falsa sensação de segurança.

Mutation testing — Stryker

Como saber se o teste **presta**? O Stryker **muta** seu código e roda os testes:

```
troca + por - · > por >= · remove uma linha · true → false
```

- teste **falha** com a mutação →  matou o mutante (bom)
- teste **passa** com a mutação →  mutante sobreviveu (teste cego ali)

```
npx stryker run      # mutation score = % de mutantes mortos
```

Mutation score mede a *qualidade* do teste — cobertura só mede o *alcance*. Aprofundamos na Aula 6 + módulo de QA EAD.

O código que vamos testar (já existe)

Antes de escrever teste, **leia o que vai testar**. Ex: `favoritesStore` (Zustand):

```
export const useFavoritesStore = create((set, get) => ({
  ids: [],
  add: (id) => set((s) => ({ ids: [...s.ids, id] })),
  remove: (id) => set((s) => ({ ids: s.ids.filter((x) => x !== id) })),
  toggle: (id) => /* add se não tem, remove se tem */,
  isFavorite: (id) => get().ids.includes(id),
}));
```

O **SUT** (System Under Test). Você não implementa — você prova que ele se comporta. Pergunta-chave: *"que entradas e estados eu preciso cobrir?"*

Hands-on — o app que vamos testar

Catálogo de filmes (RN + Expo). Lista filmes populares de uma API pública, tela de detalhe, marcar favoritos. Já vem **implementado** — você só escreve os testes.

Vamos testar 4 frentes (sem nem rodar o app):

- `favoritesStore` · `counterStore` — estado global (Zustand)
- `posterUrl` — helper que monta URL da imagem
- `isTokenError` — classifica erro de autenticação da API
- **MovieCard** — teste de tela (RNTL): renderiza e responde ao toque

```
git clone https://github.com/jacksonsmith/puc-iec-testes-aplicacoes-mobile.git
cd puc-iec-testes-aplicacoes-mobile/exercicios/02-setup-suite-unitaria/starter
npm install && npm test # posterUrl já passa verde
```

Hands-on — roteiro (90min)

1. `npm test verde` + ler `posterUrl.test.ts` (modelo) — 5min
2. `favoritesStore` — escrever os 6 testes juntos — 15min
3. **MovieCard (RNTL)** — render do título/nota + `press` → navega — **25min** ★
4. `isTokenError` — função pura da camada data — 10min
5. `counterStore` — vocês fazem sozinhos — 10min
6. Cobertura `npm run test:coverage` → ler relatório — 10min
7. Bônus: `fetchPopularMovies` com `jest.mock` da api — 5min

Pitfalls — unit testing

- ✗ **Sleep em unit test** — anti-pattern absoluto
- ✗ **Mock de tudo** — testes acoplam à implementação, não ao comportamento
- ✗ **Test names sem nada explicativo** (`testFunc1`) — rename pra revelar intenção
- ✗ **Coverage como métrica única** — alta cobertura com mutation score baixo é ilusão
- ✗ **Teste sem `expect`** — passa verde, não prova nada

Atividade 2: Suíte Unitária (10 pts)

Entrega: até 11/06. Fork do starter + sua suíte de testes.

Entregar:

1. `favoritesStore` — 6 testes (add/remove/toggle/isFavorite/clear)
2. `MovieCard` (RNTL) — teste de tela: render + toque navega ★
3. `isTokenError` (data) — 5 testes
4. `counterStore` — 3 testes
5. Cobertura $\geq 70\%$ em `src/store` e `src/utils`
6. **Eliminatório:** `npm install && npm test` roda em < 15min
7. **Bônus (+1):** `fetchPopularMovies` com `jest.mock` da api

Enunciado completo no repo: `exercicios/02-setup-suite-unitaria/enunciado.md`

Leituras obrigatórias (pré-Aula 3)

- Knott — *Hands-On Mobile App Testing*, cap 5–6
- Meszaros, G. — *xUnit Test Patterns* (Addison-Wesley 2007), cap 5
- Khorikov, V. — *Unit Testing: Principles, Practices, and Patterns* (Manning 2020), cap 4
- Bach, J. — *Session-Based Test Management* (artigo seminal)

Próxima aula (08/06)

Aula 3 — E2E em RN (Detox + Maestro)

Hoje testamos tela com RNTL (componente). Aula 3 sobe pra **E2E** — app rodando de verdade, fluxo ponta a ponta.

Pré-requisito:

- Atividade 2 entregue (suíte verde, incluindo MovieCard)
- Starter rodando `npm test`

Obrigado!

 jackson.96@gmail.com

 [linkedin.com/in/3jacksonsmith](https://www.linkedin.com/in/3jacksonsmith)

 github.com/jacksonsmith

Próxima segunda, 08/06, 19:00